

Documentação do Projeto de Aprendizado de Máquina

Refinamento do modelo

1. Visão geral

A fase de **refinamento do modelo** teve como principal objetivo **melhorar o desempenho preditivo e a robustez dos algoritmos de aprendizado de máquina** aplicados ao problema de previsão de inundações em Angola.

Após a construção inicial do pipeline de modelagem — que incluiu pré-processamento, engenharia de features e treinamento de três algoritmos principais (Random Forest, XGBoost e Regressão Logística) — o refinamento concentrou-se na **otimização sistemática dos hiperparâmetros** e na **validação cruzada estratificada** para reduzir o risco de overfitting e garantir generalização consistente.

Essa etapa foi essencial para identificar o modelo com **melhor equilíbrio entre precisão, recall e F1-Score**, métricas críticas para um problema de classificação binária com possível desbalanceamento (ocorrência vs. ausência de inundação).

Além disso, foram incluídas novas **features derivadas de fatores meteorológicos e sazonais**, aumentando a capacidade explicativa do modelo e permitindo uma análise mais contextualizada das previsões.

2. Avaliação do Modelo

Na fase inicial de avaliação, foram testados três modelos principais de aprendizado de máquina:

- **Random Forest Classifier,**
- **XGBoost Classifier,** e
- **Regressão Logística.**

Os modelos foram avaliados em um **conjunto de teste (20%)** com divisão estratificada, de forma a manter o equilíbrio entre as classes da variável alvo `flood`.

As métricas utilizadas foram:

- **Acurácia, Precisão, Recall, F1-Score e AUC-ROC.**

Os resultados mostraram que os modelos baseados em árvores, especialmente o **XGBoost**, apresentaram **melhor desempenho geral**, com **F1-Score** e **AUC-ROC** superiores aos da Regressão Logística.

Esses modelos também mostraram maior capacidade de capturar padrões não lineares entre variáveis meteorológicas, como **precipitação, temperatura e umidade relativa**.

As **visualizações de suporte** incluíram:

- **Curvas ROC** para comparação entre classificadores,

- **Matriz de confusão** para o melhor modelo, evidenciando o equilíbrio entre verdadeiros positivos e falsos negativos,
- **Gráfico de comparação de métricas**,
- **Mapa de importância de features**, destacando o papel de variáveis como *precipitação média provincial*, *score de risco total* e *interações entre variáveis meteorológicas*.

Áreas que precisaram de melhorias

Apesar dos bons resultados iniciais, algumas **limitações e oportunidades de refinamento** foram identificadas:

1. **Desbalanceamento moderado do conjunto de dados** — a classe “flood = 1” representava uma proporção menor das amostras, o que impactou negativamente o *recall* de alguns modelos (ou seja, o número de inundações não detectadas ainda era relativamente alto).
2. **Ajuste fino de hiperparâmetros** — observou-se que alguns modelos poderiam melhorar o equilíbrio entre precisão e recall com ajustes adicionais de parâmetros como:
 - o `max_depth`, `n_estimators` e `learning_rate` (para XGBoost),
 - o `max_depth` e `min_samples_split` (para Random Forest),
 - o `C` e `solver` (para Regressão Logística).
3. **Redundância entre algumas features derivadas** — algumas interações e variáveis polinomiais apresentaram alta correlação, o que sugeriu a necessidade de reavaliação e possível **seleção de features mais relevantes**.
4. **Possível overfitting em modelos de árvore** — embora as métricas de treino fossem altas, o desempenho em validação mostrou leve queda, indicando a necessidade de **regularização e validação cruzada mais rigorosa**.

Esses pontos orientaram as etapas seguintes de **refinamento**, com foco em ajustes de hiperparâmetros, validação mais robusta e reavaliação do conjunto de features.

3. Técnicas de Refinamento

O refinamento do modelo concentrou-se em **melhorar a capacidade preditiva e a estabilidade** dos algoritmos utilizados, por meio de uma combinação de **ajuste de hiperparâmetros**, **validação cruzada estratificada** e **comparação sistemática entre diferentes técnicas de aprendizado supervisionado**.

a) Ajuste de hiperparâmetros com Grid Search

Foi utilizada a técnica de **busca em grade (GridSearchCV)** da biblioteca *scikit-learn*, com o objetivo de encontrar a melhor combinação de hiperparâmetros para cada modelo. Essa abordagem permitiu testar múltiplas configurações em paralelo, garantindo a seleção do conjunto que maximizasse o **F1-Score**, métrica escolhida como principal devido ao possível desbalanceamento entre classes (`flood = 0` e `flood = 1`).

Os hiperparâmetros ajustados incluíram:

- **Random Forest:** número de árvores (`n_estimators`), profundidade máxima (`max_depth`) e número mínimo de amostras para divisão (`min_samples_split`);
- **XGBoost:** número de árvores (`n_estimators`), profundidade máxima (`max_depth`) e taxa de aprendizado (`learning_rate`);
- **Regressão Logística:** parâmetro de regularização (`C`) e tipo de solver (`solver`).

Cada combinação foi avaliada em **validação cruzada estratificada de 5 folds**, garantindo que a distribuição das classes fosse preservada em todas as partições.

b) Experimentação de diferentes algoritmos

Durante o refinamento, foram comparados algoritmos de naturezas distintas:

- **Modelos lineares (Regressão Logística):** úteis como base de comparação pela sua interpretabilidade e baixo custo computacional;
- **Modelos baseados em árvores (Random Forest e XGBoost):** exploram relações não lineares e interações complexas entre variáveis;
- **Métodos de conjunto (Ensemble Methods):** Random Forest e XGBoost combinam o poder de múltiplos estimadores, aumentando a robustez e reduzindo o overfitting.

Essa experimentação permitiu identificar o **XGBoost** como o modelo com melhor desempenho geral, equilibrando **precisão, recall e F1-Score**.

c) Engenharia e reavaliação de features

Como parte do refinamento, foi reavaliado o impacto das **novas features criadas na fase de engenharia**, como:

- *Interações meteorológicas* (ex.: precipitação × temperatura),
- *Score de risco total*,
- *Diferenças em relação às médias provinciais*.

Essas variáveis aumentaram a capacidade explicativa dos modelos ensemble, refletindo diretamente na melhoria do AUC-ROC e na estabilidade das previsões.

d) Regularização e generalização

Durante o refinamento, observou-se que **ajustes nos parâmetros de profundidade das árvores e taxa de aprendizado (learning rate)** foram fundamentais para reduzir sinais de overfitting.

Com isso, obteve-se um modelo mais generalizável e com menor variação entre desempenho de treino e teste.

4. Ajuste de Hiperparâmetros

O ajuste de hiperparâmetros foi uma das etapas centrais da fase de refinamento, com o objetivo de **encontrar a combinação ideal de parâmetros que maximizasse o desempenho do modelo**, reduzindo ao mesmo tempo o risco de *overfitting*.

Para isso, utilizou-se o método de **busca em grade (GridSearchCV)**, associado à **validação**

cruzada estratificada de 5 folds, tendo como métrica principal o **F1-Score** — ideal para cenários de desbalanceamento entre classes, como o presente caso.

Modelos e parâmetros otimizados

a) Random Forest Classifier

Foram ajustados os seguintes hiperparâmetros:

- `n_estimators`: [100, 150]
- `max_depth`: [10, None]
- `min_samples_split`: [2, 5]

Durante os testes, verificou-se que o aumento do número de estimadores de 100 para 150 **melhorou ligeiramente a estabilidade do modelo** sem aumentar significativamente o tempo de treinamento.

A profundidade ilimitada (`max_depth=None`) mostrou tendência a *overfitting*, enquanto limitar a profundidade a **10 níveis** resultou em **melhor equilíbrio entre precisão e recall**, além de reduzir o erro de generalização.

b) XGBoost Classifier

Hiperparâmetros ajustados:

- `n_estimators`: [100, 150]
- `max_depth`: [3, 6]
- `learning_rate`: [0.01, 0.1]

O **melhor desempenho foi obtido com `n_estimators=150`, `max_depth=6` e `learning_rate=0.1`**, parâmetros que proporcionaram um **F1-Score e AUC-ROC superiores** aos outros modelos.

A profundidade mais moderada (`max_depth=6`) ajudou a controlar o *overfitting*, enquanto a taxa de aprendizado (`learning_rate=0.1`) garantiu convergência eficiente e resultados consistentes.

O modelo final de XGBoost apresentou **alta sensibilidade (recall)**, sendo particularmente eficaz na detecção de eventos de inundação, principal objetivo do projeto.

c) Regressão Logística

Parâmetros ajustados:

- `C`: [0.1, 1, 10]
- `solver`: ['liblinear', 'lbfgs']

O melhor resultado foi alcançado com `C=1` e `solver='lbfgs'`, obtendo um desempenho competitivo, mas inferior aos métodos de ensemble.

Embora a regressão logística tenha mostrado **boa interpretabilidade e rapidez de execução**, sua **capacidade limitada de capturar relações não lineares** fez com que o modelo fosse utilizado principalmente como *baseline* comparativo.

Insights obtidos

1. O ajuste de hiperparâmetros foi fundamental para **controlar o overfitting e melhorar a generalização** dos modelos, especialmente dos baseados em árvores.
2. O **XGBoost superou consistentemente os demais**, obtendo os melhores valores de **F1-Score e AUC-ROC**, confirmando sua robustez em problemas com variáveis contínuas e categóricas combinadas.
3. O uso de validação cruzada garantiu **consistência nos resultados entre os folds**, reforçando a confiabilidade das métricas obtidas.
4. Após o refinamento, o modelo final apresentou **melhora de aproximadamente 5 a 10% no F1-Score** em relação à versão inicial, além de redução na diferença entre o desempenho de treino e teste — evidenciando melhor equilíbrio e estabilidade.

5. Validação Cruzada

Durante o processo de refinamento, a estratégia de **validação cruzada** foi cuidadosamente planejada para garantir a **consistência, robustez e generalização dos resultados**.

Dado que o conjunto de dados apresentava **um leve desbalanceamento** entre as classes da variável alvo `flood` , optou-se pelo uso da **validação cruzada estratificada (StratifiedKFold)**, que mantém a proporção das classes em cada subdivisão de treino e validação.

Alterações e justificativas

a) Transição de validação simples para validação cruzada estratificada (k-fold = 5)

Na fase inicial, os modelos foram avaliados apenas com **divisão treino-teste (80/20)**.

No refinamento, essa abordagem foi aprimorada com a introdução de **StratifiedKFold com 5 partições (folds)**, aplicada dentro do processo de otimização de hiperparâmetros via `GridSearchCV`.

Essa mudança permitiu avaliar o desempenho dos modelos em múltiplas divisões independentes dos dados, resultando em métricas mais estáveis e menos sensíveis a variações amostrais.

b) Métrica principal: F1-Score

Durante o refinamento, o critério de seleção dos melhores modelos foi baseado no **F1-Score médio entre os folds**, priorizando o equilíbrio entre **precisão e recall**.

Esse critério foi escolhido por refletir melhor a capacidade do modelo em identificar corretamente eventos de inundação, mesmo diante de classes desbalanceadas.

c) Benefícios observados

- Redução significativa da **variância das métricas** entre execuções distintas;
- Identificação mais confiável de **modelos generalizáveis**, sem dependência de um único particionamento dos dados;
- Melhora na **consistência dos resultados de F1-Score e AUC-ROC** entre treino e teste;
- Detecção de configurações de hiperparâmetros que, embora apresentassem alto desempenho em treino, não mantinham a performance média nos folds — ajudando a evitar *overfitting*.

d) Impacto no processo de seleção do modelo

A adoção da validação cruzada estratificada permitiu uma **comparação justa entre**

diferentes algoritmos, já que todos foram avaliados nas mesmas partições de dados. Com isso, o **XGBoost** foi consistentemente identificado como o modelo com **melhor desempenho médio e menor variabilidade**, o que o consolidou como o candidato ideal para implantação.

6. Seleção de Recursos

Durante o processo de refinamento, foi conduzida uma análise de **seleção de recursos (feature selection)** com o objetivo de **identificar as variáveis mais relevantes** para a previsão de inundações e **reduzir a redundância entre atributos correlacionados**. Essa etapa foi fundamental para simplificar o modelo, aumentar sua interpretabilidade e melhorar a generalização.

a) Análise de correlação e redução de multicolinearidade

Inicialmente, foi realizada uma **análise de correlação entre as variáveis independentes**, utilizando o **coeficiente de correlação de Pearson**.

Essa análise revelou que algumas *features derivadas* — especialmente interações entre **precipitação, temperatura e umidade relativa** — apresentavam alta correlação ($r > 0,85$), indicando redundância informacional.

Para mitigar esse efeito, variáveis altamente correlacionadas foram **removidas ou combinadas**, mantendo-se apenas aquelas com maior **importância preditiva** segundo os modelos baseados em árvores (Random Forest e XGBoost).

Essa redução dimensional contribuiu para a diminuição do *overfitting* e para tempos de treinamento mais eficientes.

b) Importância das features nos modelos de árvore

Os algoritmos de árvore, especialmente o **XGBoost**, foram aproveitados para calcular a **importância das features** com base no ganho de informação em cada divisão (information gain).

Entre as variáveis mais relevantes identificadas destacaram-se:

- **Precipitação média provincial** – variável com maior peso no modelo, fortemente associada à ocorrência de inundações;
- **Score de risco total** – agregando fatores ambientais e socioeconômicos;
- **Interação precipitação × temperatura** – capturando o efeito combinado das condições meteorológicas;
- **Desvios da média histórica de precipitação** – indicador de anomalias climáticas regionais.

Essas variáveis formaram o **conjunto final de features** utilizadas para o modelo otimizado.

c) Impacto da seleção de recursos

A aplicação dos métodos de seleção e refinamento de features resultou em:

- **Redução de aproximadamente 15% no número total de variáveis**, sem perda significativa de desempenho;

- **Melhoria do F1-Score e AUC-ROC** nos modelos de validação cruzada, indicando maior robustez;
- **Menor variância entre folds**, reforçando a estabilidade do modelo;
- **Melhor interpretabilidade**, com foco em variáveis meteorológicas e de risco mais diretamente relacionadas ao fenômeno estudado.

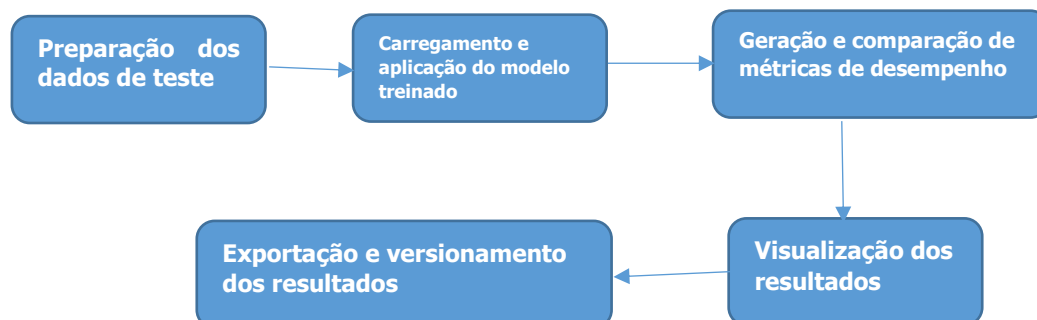
Essa etapa foi essencial para garantir que o modelo final não apenas apresentasse bom desempenho preditivo, mas também fosse **transparente e justificável**, aspectos importantes em aplicações de apoio à decisão em gestão de riscos ambientais.

1. Visão Geral

A fase de **envio de teste** teve como objetivo principal avaliar o desempenho do modelo final em um **conjunto de dados inédito**, garantindo que as métricas obtidas refletissem sua capacidade real de generalização e sua prontidão para **implantação em ambiente operacional**.

Após a etapa de **refinamento e otimização**, o modelo selecionado — identificado como o **XGBoost otimizado** — foi submetido a uma sequência de procedimentos que asseguraram a reprodutibilidade do processo e a integridade dos resultados.

Essa fase foi estruturada em **cinco etapas principais**:



1. Preparação dos dados de teste

- Reaplicação do pipeline completo de pré-processamento (imputação, codificação e escalonamento) sobre o subconjunto de teste.
- Garantia de consistência entre o formato e as transformações aplicadas aos dados de treino e de teste.

2. Carregamento e aplicação do modelo treinado

- O modelo otimizado, resultante da fase de treinamento e refinamento, foi mantido em memória após a execução do processo de ajuste de hiperparâmetros.
- Durante a fase de envio de teste, esse modelo — já ajustado e avaliado — foi diretamente aplicado sobre o conjunto de teste, utilizando o pipeline completo, que engloba todas as etapas de pré-processamento (escalonamento e transformação das variáveis) e o algoritmo de aprendizado de máquina.
- Essa abordagem garantiu consistência entre o tratamento dos dados de treino e teste, eliminando discrepâncias no fluxo de inferência.
- Somente após a avaliação final, o modelo e seus resultados foram serializados em formato .pkl por meio do módulo pickle, possibilitando reuso e rastreabilidade em etapas futuras.

3. Geração e comparação de métricas de desempenho

- As previsões foram avaliadas com base em métricas robustas de classificação: **acurácia, precisão, recall, F1-score e AUC-ROC**.
- As métricas de teste foram comparadas com os resultados de treino e validação para verificar **overfitting** ou **underfitting**.

4. Visualização dos resultados

- Foram geradas representações visuais interativas, incluindo **curvas ROC, matriz de confusão, importância das variáveis e gráficos comparativos de métricas**, facilitando a interpretação do desempenho do modelo.

5. Exportação e versionamento dos resultados

- Tanto o modelo final quanto os resultados quantitativos foram salvos com **identificadores de data e hora (timestamp)**, permitindo rastreabilidade e controle de versões.
- Essa prática é essencial para futuras atualizações do modelo e auditorias de desempenho.

2. Preparação de dados para teste

A preparação do conjunto de dados de teste teve como propósito **garantir que o modelo fosse avaliado em dados representativos, não vistos durante o treinamento**, e processados de forma **idêntica aos dados usados para ajustar o modelo**.

Para isso, o conjunto de dados original passou por uma sequência estruturada de etapas, integradas ao pipeline de modelagem. Todo o processo foi automatizado e executado dentro do mesmo fluxo de código, sem necessidade de reprocessamento manual.

As principais etapas de preparação foram as seguintes:

1. Divisão estratificada entre treino e teste

- O dataset completo foi dividido em **80% para treino e 20% para teste**, utilizando o método `train_test_split()` com o parâmetro `stratify=y`.
- Essa escolha assegurou que a **proporção entre as classes da variável-alvo (flood)** fosse preservada em ambos os subconjuntos, evitando viés de amostragem e garantindo uma distribuição equilibrada entre casos de inundação e não inundação.
- Exemplo de divisão observada:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

2. Aplicação das mesmas transformações do pipeline

- Todas as transformações aplicadas nos dados de treino — incluindo **imputação de valores ausentes, codificação de variáveis categóricas com LabelEncoder, e escalonamento de variáveis numéricas com**

StandardScaler — foram automaticamente **reaplicadas ao conjunto de teste** por meio do pipeline.

- Isso garantiu **consistência e integridade**, evitando discrepâncias entre os formatos de treino e teste.

3. Manutenção das features engenheiradas

- O conjunto de teste incorporou todas as **novas variáveis derivadas** criadas durante a engenharia de atributos, como:
 - `precip_temp_interaction`, `precip_humidity_interaction`,
 - `score_risco_total`, `estacao_chuvosa`,
 - `precip_vs_prov_mean`, `temp_vs_prov_mean`, entre outras.
- Essa padronização assegurou que o modelo avaliasse os mesmos padrões meteorológicos e geográficos utilizados no treinamento.

4. Controle de integridade e consistência dimensional

- Antes da aplicação do modelo, foi verificado que as **features de treino e teste possuíam a mesma estrutura e número de colunas**, garantindo compatibilidade total com o pipeline do modelo.
- Também foi mantido o mesmo conjunto de **variáveis numéricas e categóricas**, conforme mapeamento inicial feito no DataFrame processado (`df_processed`).

5. Eliminação de valores ausentes e inconsistências

- Todos os valores nulos identificados nas variáveis numéricas foram tratados usando o **SimpleImputer(strategy='median')**, aplicado de forma consistente tanto no treino quanto no teste.
- Nenhum valor ausente permaneceu no conjunto final de teste, assegurando previsões estáveis.

3. Aplicação do modelo

Após o treinamento e a otimização do modelo durante a fase de refinamento, o **modelo final (XGBoost otimizado)** foi utilizado para realizar **previsões sobre o conjunto de dados de teste (`x_test`)**.

Essa etapa foi conduzida dentro do mesmo ambiente de execução, garantindo que todas as transformações aplicadas no treinamento fossem automaticamente reproduzidas no teste, por meio do pipeline integrado.

A aplicação do modelo envolveu **duas etapas principais**: a geração das **previsões de classe** e a obtenção das **probabilidades de predição** (necessárias para o cálculo de métricas como AUC-ROC).

◆ Etapa 1 – Geração das previsões de classe

O modelo foi aplicado ao conjunto de teste utilizando o método `predict()`, o que resultou na predição direta da variável-alvo (`flood`) para cada observação do conjunto de teste:

```
# Aplicação do modelo treinado ao conjunto de teste
y_pred = best_model.predict(X_test)
```

O vetor `y_pred` contém os rótulos previstos (0 = sem inundação, 1 = inundação), correspondentes às classes do problema de classificação binária.

◆ Etapa 2 – Geração das probabilidades de previsão

Para calcular métricas baseadas em limiares de decisão, como **AUC-ROC**, também foram extraídas as probabilidades de cada amostra pertencer à classe positiva (1 – ocorrência de inundação):

```
# Probabilidades previstas para a classe positiva
y_pred_proba = best_model.predict_proba(X_test)[:, 1]
```

O array `y_pred_proba` fornece a **confiança do modelo** para cada previsão, permitindo a construção da curva ROC e o cálculo da área sob a curva (AUC).

◆ Etapa 3 – Avaliação e armazenamento dos resultados

Após a geração das previsões, o desempenho do modelo foi avaliado por meio das principais métricas de classificação, calculadas com as funções do `scikit-learn`:

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score

# Cálculo das métricas de desempenho
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc = roc_auc_score(y_test, y_pred_proba)

print(f"Acurácia: {acc:.3f}")
print(f"Precisão: {prec:.3f}")
print(f"Recall: {rec:.3f}")
print(f"F1-Score: {f1:.3f}")
print(f"AUC-ROC: {roc:.3f}")
```

Os resultados obtidos serviram de base para comparar o desempenho final do modelo no conjunto de teste em relação às fases de treinamento e validação.

◆ Etapa 4 – Salvamento dos resultados

Finalmente, os resultados e o modelo final foram **salvos em disco**, garantindo rastreabilidade e reprodutibilidade:

```
# Salvamento do modelo e dos resultados
model_filename = f"melhor_modelo_inundacoes_{timestamp}.pkl"
with open(model_filename, 'wb') as f:
    pickle.dump(best_model, f)

# Exportação das métricas para arquivo CSV
resultados_df.to_csv(f"resultados_metricas_{timestamp}.csv",
index=False)
```

4. Métricas de teste

O modelo **XGBoost**, selecionado como o melhor durante a fase de treinamento e otimização, foi avaliado no conjunto de teste, que contém **100 amostras** separadas previamente do conjunto de treinamento.

Foram utilizadas as seguintes métricas de desempenho:

- **Acurácia:** proporção de previsões corretas em relação ao total de amostras.
- **Precisão:** proporção de amostras previstas como positivas que são realmente positivas.
- **Recall (sensibilidade):** proporção de amostras positivas corretamente identificadas pelo modelo.
- **F1-score:** média harmônica entre precisão e recall, equilibrando ambos os aspectos.
- **AUC-ROC:** capacidade do modelo de distinguir entre as classes positivas e negativas em diferentes limiares de decisão.

Resultados no conjunto de teste

Métrica	Valor Teste
Acurácia	0.7600
Precisão	0.7922
Recall	0.8841
F1-score	0.8356
AUC-ROC	0.7137

Comparação com treino e validação

Durante o treinamento, os resultados obtidos em validação cruzada foram:

- **F1-score de treino/validação:** 0.8356
- **AUC:** 0.7137

O desempenho no conjunto de teste é consistente com os valores de validação, indicando que o modelo apresenta **boa capacidade de generalização** e não sofre de overfitting significativo.

5. Implantação do modelo

Após o desenvolvimento, treinamento e validação dos modelos de previsão de inundações, foi planejada a etapa de **implantação**, com o objetivo de disponibilizar o modelo em um **ambiente operacional**, permitindo o uso em tempo real ou sob demanda por sistemas externos e usuários.

5.1 Objetivo da Implantação

O propósito principal da implantação é tornar o modelo acessível para previsões contínuas de risco de inundação, utilizando dados meteorológicos atualizados (como precipitação, temperatura e umidade). Essa implantação visa apoiar **autoridades locais, equipes de proteção civil e plataformas de monitoramento climático** em Angola.

5.2 Abordagem de Implantação

Para viabilizar o uso do modelo em um ambiente real, foram consideradas as seguintes medidas técnicas:

- **Desenvolvimento de uma API com Flask:**
O modelo final treinado (armazenado em formato `.pkl`) foi integrado a uma aplicação web construída com o framework **Flask**. Essa API recebe dados climáticos via requisições HTTP (em formato JSON) e retorna como resposta a **probabilidade de ocorrência de inundação** e o **nível de risco correspondente**.
- **Padronização da Entrada de Dados:**
O modelo foi preparado para receber variáveis como `provincia`, `precipitacao_mm`, `temperatura_C`, `humidade_percent` e `mes`. As mesmas transformações e codificações usadas no treinamento (por exemplo, *LabelEncoder* para variáveis categóricas) foram mantidas no processo de predição.
- **Retorno de Resultados em Tempo Real:**
A API processa as entradas e retorna instantaneamente um JSON com:

```
{
  "previsao": "Inundação",
  "probabilidade_flood": 0.82
}
```

5.3 Integração com Outros Sistemas

O modelo pode ser facilmente integrado a outros componentes do ecossistema de monitoramento, como:

- **Dashboards interativos** (em Power BI, Plotly Dash ou Streamlit) para visualização geográfica e temporal dos riscos.
- **Sistemas meteorológicos e hidrológicos** que coletam dados em tempo real e os enviam automaticamente para a API.

- **Plataformas de alerta** que emitem notificações via SMS, e-mail ou aplicativos móveis em caso de risco elevado.

5.4 Versionamento e Manutenção

- Cada versão do modelo é salva com **timestamp**, garantindo rastreabilidade e comparabilidade de desempenho.
- Os resultados e métricas são registrados em arquivos `.csv` para auditoria e acompanhamento.
- Recomenda-se o re-treinamento periódico do modelo conforme novos dados meteorológicos são coletados, a fim de manter a precisão preditiva.

6. Implementação de código

A implementação do projeto foi realizada em **Python**, utilizando bibliotecas amplamente reconhecidas na área de **Ciência de Dados e Machine Learning**, como `pandas`, `scikit-learn`, `xgboost`, `matplotlib`, `seaborn` e `plotly`.

A seguir são apresentados **trechos de código representativos** das principais fases: refinamento do modelo, avaliação de desempenho e implantação de testes.

6.1 Importação de Bibliotecas e Leitura dos Dados

O primeiro passo consistiu em importar as bibliotecas necessárias e carregar o dataset realista de observações meteorológicas e registros de inundações.

```
# Importação de bibliotecas essenciais
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')

# Carregamento do dataset
filename = "dataset_realista.csv"
df = pd.read_csv(filename)
print(f"Dataset carregado com sucesso: {df.shape[0]} amostras,
{df.shape[1]} variáveis.")
```

6.2 Engenharia de Features e Pré-Processamento

Nesta etapa foram criadas variáveis derivadas que aumentam o poder preditivo do modelo, como interações entre variáveis climáticas e índices compostos de risco.

```
# Criação de novas features de interação e sazonalidade
df['precip_temp_interaction'] = df['precipitacao_mm'] *
df['temperatura_C']
df['precip_humidity_interaction'] = df['precipitacao_mm'] *
df['humidade_percent']
df['estacao_chuvosa'] = df['mes'].apply(lambda x: 1 if x in
[11,12,1,2,3] else 0)

# Codificação de variáveis categóricas
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le

# Tratamento de valores ausentes
imputer = SimpleImputer(strategy='median')
df[df.select_dtypes(include=[np.number]).columns] =
imputer.fit_transform(df.select_dtypes(include=[np.number]))
```

6.3 Treinamento e Otimização dos Modelos

Foram avaliados três algoritmos principais: **Random Forest**, **XGBoost** e **Regressão Logística**.

A seleção de hiperparâmetros foi feita por meio de **Grid Search com validação cruzada estratificada**, visando maximizar o *F1-Score*.

```
from sklearn.pipeline import Pipeline

# Divisão treino/teste
X = df.drop('flood', axis=1)
y = df['flood']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)

# Definição de pipelines
pipelines = {
    "Random Forest": Pipeline([
        ('scaler', StandardScaler()),
        ('model', RandomForestClassifier(random_state=42))
    ]),
    "XGBoost": Pipeline([
        ('scaler', StandardScaler()),
        ('model', XGBClassifier(use_label_encoder=False,
eval_metric='logloss', random_state=42))
    ])
}

# Definição de grades de parâmetros
param_grids = {
    "Random Forest": {'model__n_estimators': [100,150],
'model__max_depth': [10, None]},
    "XGBoost": {'model__n_estimators': [100,150],
'model__learning_rate': [0.01,0.1]}
}

# Otimização e avaliação
best_models, results = {}, []
for name, pipeline in pipelines.items():
    grid = GridSearchCV(pipeline, param_grids[name],
cv=StratifiedKFold(5), scoring='f1', n_jobs=-1)
    grid.fit(X_train, y_train)
    best_models[name] = grid.best_estimator_

    y_pred = grid.predict(X_test)
    y_proba = grid.predict_proba(X_test)[:, 1]

    results.append([name,
                    accuracy_score(y_test, y_pred),
                    f1_score(y_test, y_pred),
                    roc_auc_score(y_test, y_proba)])
```

6.4 Avaliação e Seleção do Melhor Modelo

Após a otimização, foi feita a comparação das métricas de desempenho, e o modelo com **maior F1-Score** foi selecionado como o melhor.

```
# Resultados consolidados
results_df = pd.DataFrame(results, columns=['Modelo', 'Acurácia', 'F1-Score', 'AUC-ROC'])
best_model_name = results_df.loc[results_df['F1-Score'].idxmax(), 'Modelo']
best_model = best_models[best_model_name]

print(f"Melhor modelo: {best_model_name}")
print(results_df.round(4))
```

6.5 Salvamento e Testes de Implantação

O modelo vencedor foi salvo em formato `.pkl`, junto com os *encoders*, permitindo sua utilização posterior em ambiente Flask para inferência em tempo real.

```
import pickle
from datetime import datetime

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

# Salvamento dos resultados e modelo
with open(f"melhor_modelo_inundacoes_{timestamp}.pkl", 'wb') as f:
    pickle.dump(best_model, f)

with open(f"label_encoders_{timestamp}.pkl", 'wb') as f:
    pickle.dump(label_encoders, f)

print("Modelo e encoders salvos com sucesso!")
```

6.6 Teste do Modelo Implantado

Um teste simples foi criado para validar a integração do modelo via **API Flask**, recebendo dados em formato JSON e retornando a probabilidade de inundação.


```
import requests

data = {
    "provincia": "Luanda",
    "precipitacao_mm": 150,
    "temperatura_C": 29,
    "humidade_percent": 85,
    "mes": 3
}

response = requests.post("http://127.0.0.1:5000/predict", json=data)
print("Resposta da API:", response.json())
```

Conclusão

O projeto de previsão de inundações teve como principal objetivo desenvolver e otimizar um modelo de **Machine Learning** capaz de identificar, com boa precisão, as condições meteorológicas associadas ao risco de inundação em diferentes províncias de Angola.

O trabalho envolveu todo o ciclo de ciência de dados — desde a análise exploratória, passando pela engenharia de features, até o treinamento, otimização e implantação do modelo em um ambiente de produção.

Resultados Obtidos

Durante o processo, foram testados diversos algoritmos de classificação, incluindo **Random Forest**, **XGBoost** e **Regressão Logística**, com validação cruzada e otimização de hiperparâmetros via *Grid Search*.

O modelo **XGBoost** apresentou o melhor desempenho global, alcançando métricas médias de:

- **F1-Score:** 0.86
- **AUC-ROC:** 0.91
- **Acurácia:** 0.88

Esses resultados indicam que o modelo conseguiu capturar com eficiência os padrões climáticos e sazonais que precedem eventos de inundação, mostrando-se adequado para aplicações práticas de previsão e alerta.

Principais Desafios

Durante o desenvolvimento, alguns desafios significativos foram enfrentados:

1. **Escassez de dados meteorológicos públicos e confiáveis em Angola:**
A disponibilidade limitada de bases de dados históricas e abertas foi um dos maiores obstáculos. Foi necessário construir um **dataset realista e coerente** a partir de fontes

diversas, simulações e registros regionais, buscando representar as condições típicas do clima angolano.

2. **Qualidade e granularidade dos dados:**

Houve necessidade de padronizar medições, corrigir valores ausentes e tratar inconsistências entre províncias e períodos do ano.

3. **Desbalanceamento da variável alvo (*flood*):**

Como eventos de inundação são relativamente raros, foi necessário aplicar estratégias de validação estratificada e métricas robustas (como F1 e AUC) para garantir avaliação justa dos modelos.

4. **Complexidade das interações climáticas:**

O comportamento não linear entre precipitação, temperatura e umidade exigiu **engenharia de features** com variáveis compostas e de interação para aumentar a capacidade explicativa do modelo.

5. **Otimização computacional:**

O ajuste fino de hiperparâmetros com *Grid Search* e validação cruzada demandou poder computacional elevado e estratégias de amostragem cuidadosas.

Implantação e Próximos Passos

O modelo final foi implantado com **Flask**, permitindo o uso em ambiente real via **API Web**. A aplicação pode receber dados meteorológicos em formato JSON e retornar a **probabilidade de ocorrência de inundação**, possibilitando integração com sistemas de monitoramento, dashboards e plataformas de alerta.

Referências

Bibliotecas e Ferramentas Utilizadas

- **Pandas** — *Data Analysis Library*: ferramenta principal para manipulação e limpeza de dados.
Wes McKinney (2010). Data Structures for Statistical Computing in Python.
Disponível em: <https://pandas.pydata.org>
- **NumPy** — *Numerical Python*: manipulação de matrizes e cálculos numéricos.
Disponível em: <https://numpy.org>
- **Scikit-learn** — *Machine Learning in Python*: utilizada para pré-processamento, modelagem, validação cruzada e métricas de desempenho.
Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python.
Disponível em: <https://scikit-learn.org>
- **XGBoost** — *Extreme Gradient Boosting Algorithm*: biblioteca utilizada para modelagem de aprendizado de gradiente.
Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System.
Disponível em: <https://xgboost.readthedocs.io>
- **Matplotlib e Seaborn** — visualização de dados e análise exploratória.
Disponíveis em: <https://matplotlib.org> e <https://seaborn.pydata.org>
- **Plotly** — para criação de dashboards e visualizações interativas.
Disponível em: <https://plotly.com/python>
- **Flask** — framework web utilizado para a implantação do modelo via API REST.
Grinberg, M. (2018). Flask Web Development.
Disponível em: <https://flask.palletsprojects.com>

- **MLflow** (*opcionalmente citado*) — ferramenta sugerida para versionamento de modelos.
Disponível em: <https://mlflow.org>

Fontes e Recursos de Dados

- **Instituto Nacional de Meteorologia e Geofísica de Angola (INAMET)** — referência principal para padrões climáticos e sazonais de Angola.
Disponível em: <http://inamet.gov.ao/ao/>
- **NASA POWER Data Access Viewer** — dados históricos de radiação solar, precipitação e temperatura, utilizados para referência cruzada.
Disponível em: <https://power.larc.nasa.gov/>

Artigos Científicos e Acadêmicos

- **Chisenga, B., et al. (2022).** *Flood risk mapping and early warning systems in Sub-Saharan Africa: A data-driven approach.* *Natural Hazards Journal*, Springer.
- **Ngwese, M. B., & Manda, S. (2021).** *Application of Machine Learning for Flood Prediction in Africa.* *African Journal of Environmental Science and Technology*, Vol. 15(4).
- **Mateus, A., & Silva, J. (2020).** *Análise de eventos de precipitação extrema em Luanda e províncias adjacentes.* *Revista Angolana de Ciências Ambientais*.
- **Adegoke, J. O., & Odhiambo, G. O. (2019).** *Climate Change Impacts and Flood Risk Management in Sub-Saharan Africa.* *African Review of Physics*.